Practical - 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

Theory:

1. Are sessions started and destroyed securely?

   Answer: Yes, sessions are started using session_start(). After successful login, the session ID is regenerated. During logout, session data is cleared and the session is destroyed.

2. Is password_verify() used correctly?

   Answer: Yes, password_verify() is used to check whether the entered password matches the hashed password stored in the database.

3. Is unauthorized dashboard access blocked?

   Answer: Yes, unauthorized dashboard access is blocked. If a user is not logged in, they are redirected to the login page.

4. Is session timeout implemented?

   Answer: Yes, session timeout is implemented. If the user remains inactive for more than five minutes, the session expires and the user must log in again.

5. Are admin and student users redirected to appropriate dashboards?

   Answer: Yes, admin and student users are redirected to their respective dashboards according to their roles. Admin users access the admin dashboard, while student users access the student dashboard.

A. Implementation/ Output/Result **(Keep only one option which is suitable to practical)**

PHP functions

Function	Purpose
session_start()	Starts or resumes a session.
session_regenerate_id(true)	Generates a new session ID after login.
session_unset()	Removes session variables.

session_destroy()	Destroys the current session.
password_hash()	Converts a password into a secure hash.
password_verify()	Checks the entered password against its stored hash.
time()	Returns the current timestamp.
trim()	Removes extra spaces from the beginning and end of a string.
isset()	Checks whether a variable is set.
header()	Sends a redirect or other HTTP header.
exit	Stops script execution.
htmlspecialchars()	Helps prevent HTML injection when displaying user input.
ini_get()	Reads a PHP configuration setting.
setcookie()	Creates or updates a cookie.
session_get_cookie_params()	Gets the session cookie settings.
fetch_assoc()	Fetches a database row as an associative array.
execute()	Executes a prepared SQL statement.
bind_param()	Binds values to placeholders in a prepared statement.
prepare()	Prepares an SQL statement for execution.
connect_error	Provides the database connection error, if one occurs.

3. PHP statements and superglobals

Statement or variable	Purpose
$_POST	Retrieves values submitted through a POST form.
$_SESSION	Stores user information across requests.
$_SERVER	Provides server and request information.
$_COOKIE	Reads cookie values sent by the browser.
require	Includes another PHP file, such as db.php.
if	Checks a condition.
else	Runs code when the if condition is false.
foreach	Loops through array elements.
echo	Displays output.
isset()	Checks whether a variable or array key exists and is not null.
&&	Checks whether two conditions are both true.

4. MySQL and database commands

Command or method	Purpose
new mysqli()	Connects PHP to the MySQL database.

CREATE DATABASE	Creates a database.
CREATE TABLE	Creates a database table.
INSERT INTO	Adds records to a table.
SELECT	Retrieves records from a table.
UPDATE	Updates existing records.
PRIMARY KEY	Uniquely identifies each record.
AUTO_INCREMENT	Automatically generates numeric IDs.
NOT NULL	Requires a value in a column.
UNIQUE	Prevents duplicate values in a column.
VARCHAR	Stores variable-length text.
ENUM	Restricts a column to specified values, such as admin or student.
TIMESTAMP	Stores date and time information.

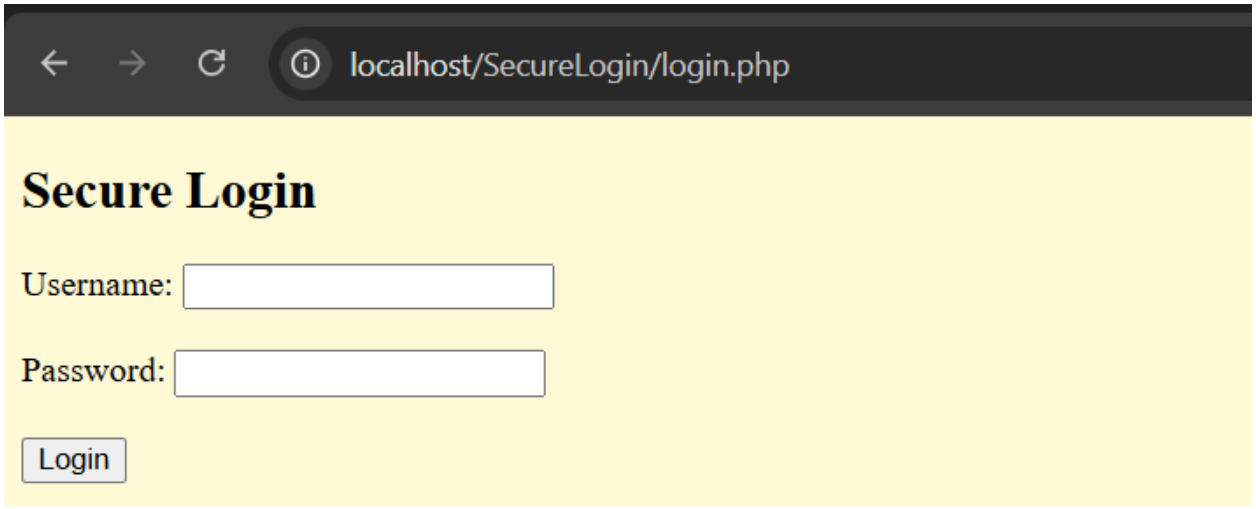
5. Authentication and security concepts

Concept	Purpose
PHP sessions	Maintain a user's login state.
Cookies	Store information in the browser.
Password hashing	Protects stored passwords from being readable as plain text.
Role-based access control	Restricts access according to the user's role.
Session timeout	Ends access after a period of inactivity.
Prepared statements	Help prevent SQL injection.
Authentication	Verifies the identity of a user.
Authorization	Determines what an authenticated user is allowed to access.
Redirection	Sends the user to the appropriate page.

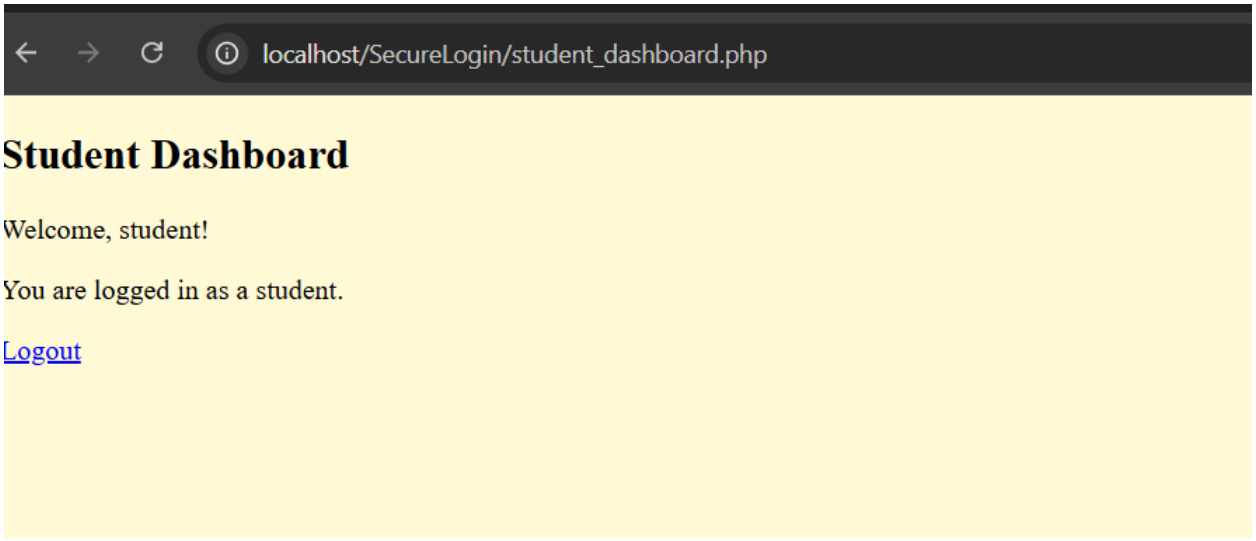
B. Evidence of Work (Screenshots)



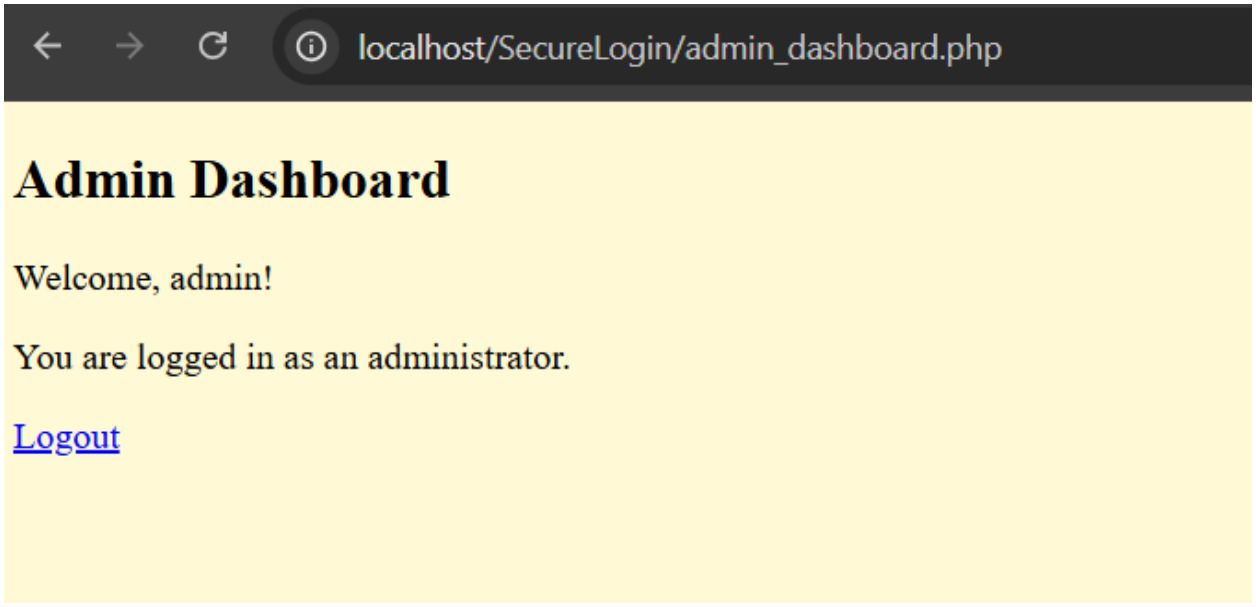
Test users created successfully!

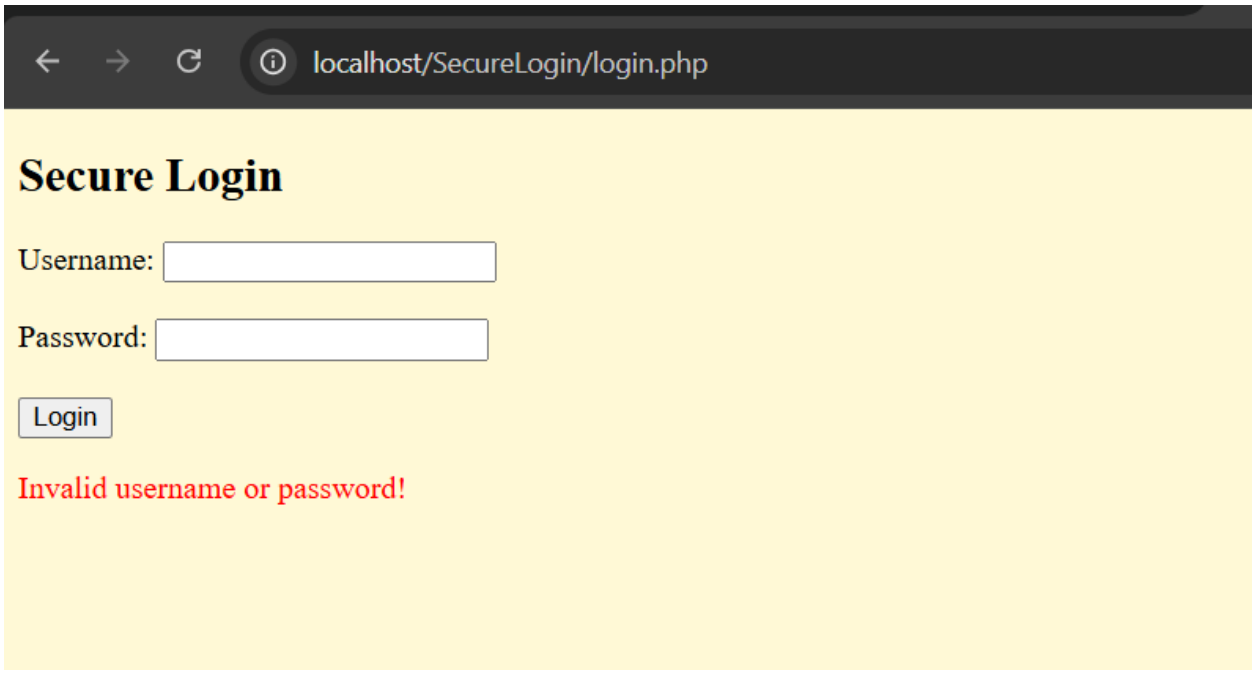


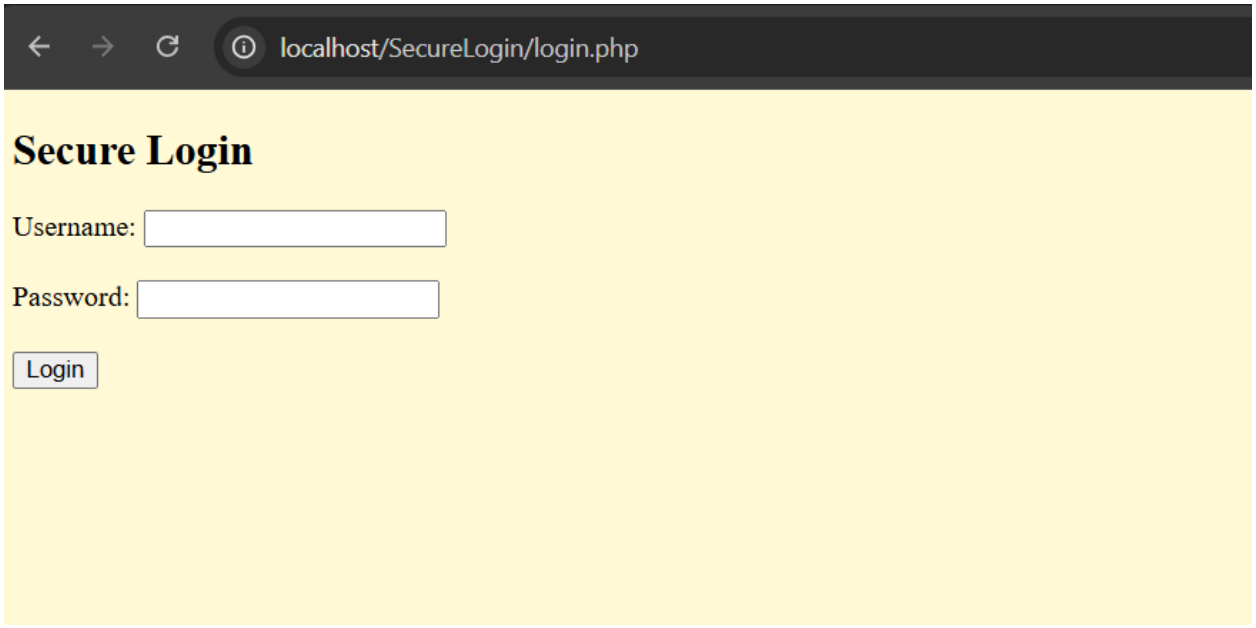
Before.



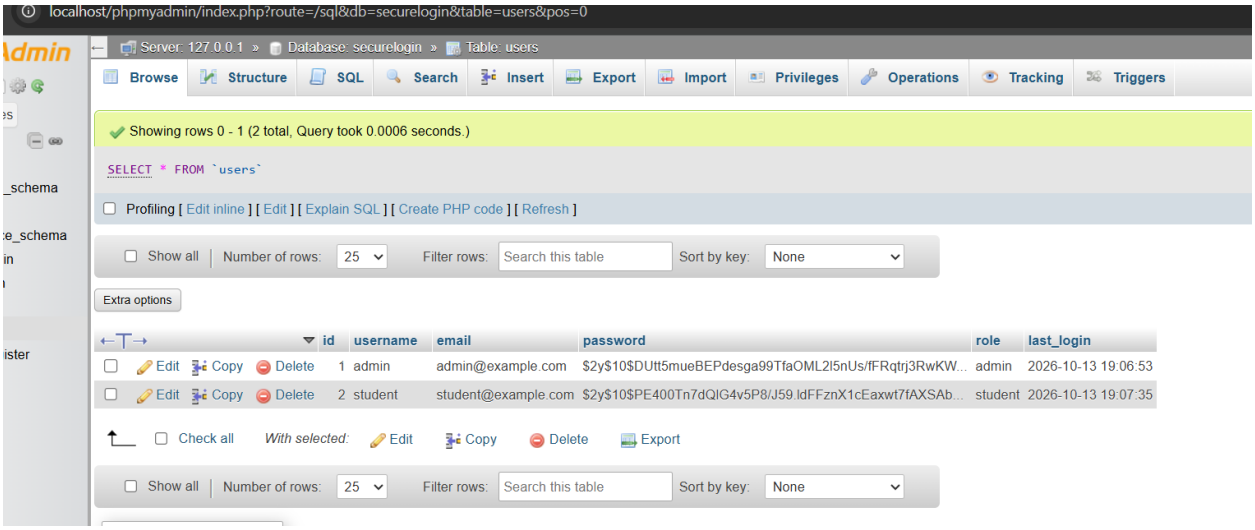
After student dashboard was created .


localhost/SecureLogin/admin_dashboard.php
Admin Dashboard
Welcome, admin!
You are logged in as an administrator.
Logout


localhost/SecureLogin/login.php
Secure Login
Username:
Password:
Login
Invalid username or password!



For unauthorized access. redirected to login.php**.**



C. GitHub Repository Link:

https://github.com/diyatolia1212/StudentHub/tree/0511c6f530f474de152ae6686e677386f45094b1/SecureLogin

D. Challenges Faced:

1. Verifying passwords using password hashing.
2. Managing sessions and regenerating session IDs after login.
3. Restricting dashboard access based on user roles.
4. Implementing logout and session timeout.

Learning Outcomes:

1. Learned to implement login and logout using PHP sessions.
2. Understood password hashing and verification.
3. Learned to prevent unauthorized dashboard access.
4. Implemented role-based redirection for admin and student users.
5. Understood session timeout and secure authentication.

Faculty Remarks: (Keep this blank for faculty)